

Relazione Progetto: Video Stabilization

Emanuele Barbagallo

I notebook di analisi e i video di confronto sono disponibili sulla repository GitHub ufficiale: [barbagalloemanuele/Video_Stabilization](https://github.com/barbagalloemanuele/Video_Stabilization).

1 Introduzione e Problema Iniziale

La stabilizzazione video è un problema fondamentale nell'ambito della *computer vision*. L'obiettivo primario è rimuovere i movimenti indesiderati della fotocamera generati durante le riprese a mano libera o in movimento, mantenendo il movimento intenzionale il più fluido possibile. Questo problema è particolarmente rilevante nel contesto odierno, caratterizzato da un uso massiccio di dispositivi portatili, smartphone, droni e action cam.

Un video instabile non solo risulta sgradevole e affaticante alla visione per l'utente, ma degrada drasticamente le performance di algoritmi di compressione video, object tracking e ricostruzione 3D. Stabilizzare un video richiede di stimare con precisione la traiettoria originale della camera, filtrarne le componenti ad alta frequenza (il tremolio) e infine deformare i frame originali per simulare una nuova traiettoria fluida. L'obiettivo del progetto è implementare e confrontare due diverse strategie di stabilizzazione classiche per compensare le trasformazioni affini tra frame consecutivi.

2 Il Dataset e l'Approccio Originale

I dati utilizzati per questo progetto provengono dal dataset pubblico **StabFR** (Video Stabilization and Assessment) creato dal gruppo di ricerca del Dr. Lei Zhang. Il dataset offre sequenze video suddivise per scenario d'uso (es. climbing, driving, riding, walking). La peculiarità fondamentale di questo dataset, e il motivo per cui è stato scelto, è l'approccio ingegnoso con cui gli autori originali hanno raccolto i dati per ottenere un vero e proprio *ground truth* per la valutazione *full-reference*. Hanno utilizzato un dispositivo personalizzato che montava due fotocamere accoppiate fianco a fianco:

- Una fotocamera **GoPro**, montata rigidamente sul supporto, per catturare il video originale shaky.
- Una fotocamera **DJI Osmo**, dotata nativamente di una stabilizzazione meccanica e motorizzata tramite *gimbal*, per catturare in parallelo la versione già stabilizzata (la ground truth).

Nel loro studio originale, gli autori hanno affrontato il problema di regolarizzazione proponendo una sofisticata tecnica di *Geodesic Video Stabilization*, operando nello spazio delle trasformazioni geometriche (Lie group manifold) e smussando il movimento tramite una metrica Riemanniana. Il loro tool software si occupava proprio di misurare quanto una determinata traiettoria risultasse curva a livello geodetico per poterne decretare la stabilità.

Per le finalità di questo progetto ci si è focalizzati su un approccio sperimentale di Computer Vision "classica", basato sul tracciamento diretto del movimento e sull'applicazione di filtri temporali alle trasformazioni affini. Il dataset con la ripresa DJI Osmo ci ha quindi fornito il *benchmark* ideale per valutare quantitativamente i nostri algoritmi software contro uno standard di stabilizzazione hardware.

3 Algoritmi Scelti e Motivazioni

Sono stati implementati due algoritmi:

1. Lucas-Kanade (Optical Flow):

- Questo metodo stima il flusso ottico (il movimento apparente dei pixel) tra frame consecutivi. Si basa sul principio della costanza dell'illuminazione spaziale, risolvendo le equazioni del moto in piccoli intorni spaziali tramite l'uso delle piramidi di immagini.
- È stato scelto perché assicura un'eccellente precisione sub-pixel. È estremamente affidabile per video caratterizzati da movimenti fluidi, regolari e con sfondi prevalentemente omogenei. Riesce inoltre a tracciare gradienti di colore dove metodi estrattori di angoli fallirebbero (come in situazioni di leggere sfocature o motion blur).

2. ORB + RANSAC (Feature Matching):

- Si tratta di un metodo sparse basato su feature geometricamente distinte. Estrae prima dei keypoints con l'algoritmo **ORB** (Oriented FAST and Rotated BRIEF), intrinsecamente invariante alle rotazioni ed efficiente a livello computazionale. I descrittori dei keypoints estratti nei due frame adiacenti vengono quindi abbinati matematicamente tramite distanza di Hamming.
- La scelta vincente qui è l'aggiunta di **RANSAC** (Random Sample Consensus) nella derivazione della matrice di trasformazione. RANSAC campiona sottoinsiemi di matching alla ricerca del modello geometrico che accordi la maggioranza degli inlier, rimuovendo gli outlier. Questo rende l'algoritmo formidabile nel gestire scene complesse, persone che attraversano l'inquadratura, automobili, ignorando il rumore locale e agganciando la stabilizzazione al puro background.

4 Workflow Adottato

Il processo di stabilizzazione è stato suddiviso in precise fasi ingegneristiche e di signal-processing all'interno dei notebook Jupyter (`LucasKanade_OpticalFlow.ipynb` e `ORB_Ransac.ipynb`):

1. Lettura e Pre-processing dei Dati:

- I video d'origine vengono letti tramite la libreria `cv2`.
- Ogni frame viene scalato (se eccessivamente largo) e convertito dalla codifica RGB allo spazio di colori a Singolo Canale (Scala di grigi) per ridurre il carico computazionale durante l'estrazione delle features e minimizzare i problemi di variazione cromatica.

2. Stima e Tracciamento del Movimento:

- Nel caso dell'Optical Flow, estraiamo i keypoints rilevanti sul primo frame usando la funzione `cv2.goodFeaturesToTrack` e li tracciamo in avanti nei frame successivi richiamando la funzione piramidale `cv2.calcOpticalFlowPyrLK`.
- Nel caso di ORB, applichiamo un detettore su ogni fotogramma per raccogliere i descrittori locali. Un `BFMatcher` (Brute-Force) crea i collegamenti tra le features omologhe in frames contigui. Successivamente la funzione `cv2.estimateAffinePartial2D` integra l'algoritmo RANSAC per fornire la matrice di rototraslazione 2×3 priva dell'inquinamento degli outlier.

3. Integrazione della Traiettoria Cumulativa:

- Una singola matrice di transizione ci dice soltanto di quanto la telecamera si è mossa da un frame al suo diretto successivo. Per comprendere lo spazio assoluto, accumuliamo iterativamente questi step ($dx, dy, d\theta$) ottenendo una curva che descrive l'intera traiettoria globale temporale della fotocamera, detta "camera trajectory".

4. Regularizzazione tramite Low-Pass Filtering (Smoothing):

- Avendo la curva della traiettoria assoluta (caratterizzata da picchi netti in caso di forte tremolio), viene applicato un filtro di media mobile longitudinale (Moving Average) su una finestra prefissata (es. radius=30-50 frame). Questo passaggio preserva le componenti a "bassa frequenza" (il vero movimento che il cameraman avrebbe voluto compiere) appiattendole le componenti ad "alta frequenza" (l'impatto dei passi o il jitter involontario delle mani).

5. Compensazione Finale (Image Warping):

- Sottraendo la traiettoria originale dalla traiettoria "smooth", si ottiene il Delta di trasformazione desiderato per il frame corrente.
- I singoli fotogrammi del video originale vengono inviati alla funzione geometrica di deformazione spaziale `cv2.warpAffine` applicando tale trasformazione correttiva. Opzionalmente, vengono eseguiti dei leggeri crop ai bordi al fine di sopprimere i margini neri causati dalla ricentatura forzata.
- Tutti i risultati vengono salvati iterativamente tramite `cv2.VideoWriter`.

5 Video di Comparazione Creati

Per fornire un'analisi qualitativa immediata, sono stati generati diversi video all'interno della cartella `data/` per permettere l'ispezione empirica:

- `comparison_shaky.mp4`: Videomontaggio side-by-side tra il filmato shaky originale e l'output stabilizzato software, per apprezzare visivamente il lavoro dell'algoritmo sull'abbattimento del tremolio.
- `comparison_groundtruth.mp4`: Videomontaggio tra il nostro video stabilizzato via software e il video catturato dallo stabilizzatore hardware (DJI Osmo), fornito come standard d'eccellenza.
- `mask.mp4`: Video di debug ad alto valore diagnostico. Mostra in sovrimpressione cosa sta interpretando l'algoritmo frame-by-frame: per LK vedremo le "code" del flusso ottico, per ORB+RANSAC assisteremo alle linee colorate che legano i matching *inlier* sopravvissuti.
- `LucasKanade_vs_ORB+RANSAC.mp4`: Un video side-by-side definitivo che riproduce direttamente i risultati finali dei due algoritmi, in modo da poter validare a colpo d'occhio chi dei due riesce ad esaltare maggiormente la fluidità percepita dell'immagine senza sacrificare eccessivamente i margini.

6 Valutazione Quantitativa e Qualitativa

L'intero processo valutativo è codificato e reso automatizzato in `Valutazione_Video.ipynb`. I calcoli si basano sia sulle informazioni spettrali che sulle derivate locali dei vettori movimento.

- **PSNR (Peak Signal-to-Noise Ratio)** e **SSIM (Structural Similarity Index)**: Misurano rispettivamente la distorsione globale in scala logaritmica (in decibel) e la fedeltà strutturale percettiva dell'output se messo in overlay perfetto con il frame *groundtruth*. Maggiore è il punteggio, più il video appare visivamente fedele e non deturpato da *crop* estesi, *blurring* o bordi disallineati.
- **Stability Score**: Metrica empirica che stima lo spettro energetico delle traslazioni (derivando le deviazioni tra frame consecutivi elaborati). Più basso è questo valore, meno il fotogramma si sposta caoticamente in alto/basso o destra/sinistra nel tempo. Un basso *Stability Score* è sinonimo di movimenti *steady-cam* simili.

6.1 Tabella dei Risultati

Cartella	PSNR_LK	PSNR_ORB	SSIM_LK	SSIM_ORB	Stability_LK	Stability_ORB	Stability_GT	Stability_Shaky	Miglior Metodo
climbing1	13.10	13.09	0.3318	0.3319	1.728002	1.542570	1.493980	8.637057	ORB+RANSAC (+82.1%)
climbing2	11.59	11.56	0.2411	0.2408	0.827392	0.700149	0.311864	19.209363	ORB+RANSAC (+96.4%)
driving1	13.58	13.52	0.4157	0.4145	14.295192	12.416973	27.810958	22.395803	ORB+RANSAC (+44.6%)
driving2	12.01	12.07	0.3939	0.3930	16.211923	18.264985	8.407414	16.817671	Lucas-Kanade (+3.6%)
riding1	16.74	16.64	0.5236	0.5213	3.430997	4.005630	2.282321	84.840079	Lucas-Kanade (+96.0%)
riding2	14.13	14.10	0.4506	0.4491	4.587041	5.730770	1.304891	63.598964	Lucas-Kanade (+92.8%)
walking1	14.51	14.54	0.4318	0.4323	0.297035	0.229085	0.016238	11.371488	ORB+RANSAC (+98.0%)
walking2	15.20	15.21	0.3961	0.3963	0.198234	0.211539	0.028066	9.571490	Lucas-Kanade (+97.9%)

Tabella 1: Risultati comparativi dei due algoritmi.

7 Conclusioni

I risultati finali dimostrano chiaramente che non esiste un algoritmo perfetto in ogni situazione, ma c'è un compromesso tra la capacità di gestire gli ostacoli e la qualità visiva del video:

- **I punti di forza di Lucas-Kanade**: Questo metodo si rivela eccellente quando la scena è abbastanza sgombra e il movimento della videocamera, pur essendo mosso, rimane continuo e senza scatti improvvisi (come nei video riding1 e riding2). In queste situazioni, calcola gli spostamenti in modo molto preciso e applica correzioni leggere che mantengono alta la qualità dell'immagine (ottenendo i migliori punteggi di PSNR e SSIM). Va in crisi, però, quando i movimenti sono troppo bruschi: se tra un frame e l'altro c'è uno scossone forte, l'algoritmo perde il tracciamento dei pixel.
- **I vantaggi di ORB + RANSAC**: Questa combinazione, invece, domina nelle scene più caotiche o con molti elementi in movimento (come climbing1, climbing2, walking1 e driving1). RANSAC è eccezionale nell'ignorare gli ostacoli temporanei (come persone o veicoli di passaggio), concentrandosi solo sul vero sfondo per calcolare il movimento. Questo permette di appiattire in modo impressionante anche i tremolii più violenti, migliorando drasticamente lo *Stability Score*. Il compromesso è che, per compensare questi scossoni, le immagini vengono ruotate o spostate maggiormente, creando dei bordi neri più ampi che abbassano leggermente la fedeltà visiva complessiva.

In sintesi, la soluzione ideale per il futuro sarebbe un'architettura **Ibrida**: usare Lucas-Kanade come metodo di base per le scene più stabili e lineari, passando il controllo a ORB+RANSAC solo nel momento in cui la telecamera subisce urti troppo forti e l'Optical Flow non riesce più a seguire la scena.